ASTANA IT UNIVERSITY • DEPARTMENT OF COMPUTER SCIENCE & SOFTWARE ENGINEERING

Course: Information and Communication Technologies (ICT101) • Instructor: Diana Kazbekova

Assignment 3 — Database Systems and Cybersecurity (Project Build, Part 1)

Project: Redesign and Cloud Optimization of AITU LMS (Moodle) Student Portal

Team Members (Group SE-2625):
• Maratuly Adilkhan (Student ID: 261173) — Lead Analyst & Defense Lead
• Belzhanov Alikhan (Student ID: 261875) — System & Database Architect
• Khamidolla Sanzhar (Student ID: 260345) — UI/UX & Prototype Lead

Submission Details:
• Checkpoint: Assignment 3 (Week 6 Practical Build)
• Database Engine: SQLite 3.42 (Local) / Cloud Firestore (Sync)
• Date: 12 October 2026

Part A — Data Model and Working Database (25 points)

A1. Entity-Relationship Diagram (ERD)

To support the requirements established in Project Stage 1 (specifically **FR1** for unified course overviews, **FR3** for aggregated deadline tracking, and **FR7** for role-based access control), our team designed a fully normalized relational data model consisting of six distinct entities. The model formally resolves the many-to-many relationship between Students and Courses through an explicit associative linking table (`enrollments`) and maintains full referential integrity via cascading constraints.

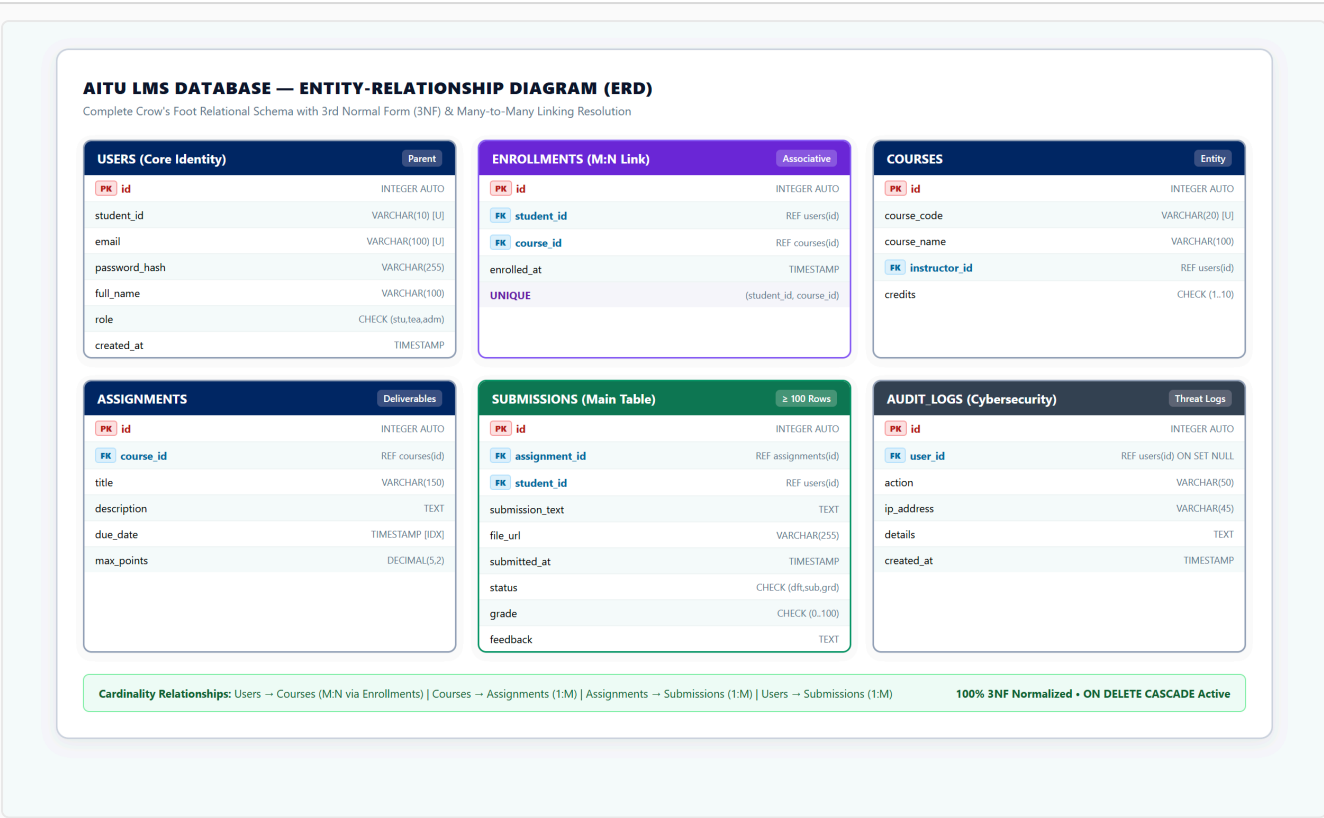


Figure 1: Complete Entity-Relationship Diagram (ERD) for the AITU LMS database showing primary keys (PK), foreign keys (FK), check constraints, and relationship cardinalities in Crow's Foot notation.

Relational Entities and Cardinality Specifications:

1. **USERS** (1:N with Enrollments, 1:N with Submissions, 1:N with Audit Logs): Holds identity and authentication credentials. Attributes: `id` (PK), `student_id` (Unique), `email` (Unique, restricted to @astanait.edu.kz), `password_hash` (Bcrypt/SHA-256), `full_name`, `role` (CHECK: student, teacher, admin), and `created_at`.
2. **COURSES** (1:N with Enrollments, 1:N with Assignments): Represents registered academic disciplines. Attributes: `id` (PK), `course_code` (Unique, e.g. 'MATH101'), `course_name`, `instructor_id` (FK referencing users.id), and `credits`.
3. **ENROLLMENTS** (Associative Linking Table): Resolves the M:N relationship between Students and Courses. Attributes: `id` (PK), `student_id` (FK), `course_id` (FK), `enrolled_at`. Enforces a `UNIQUE(student_id, course_id)` composite constraint to prevent duplicate course registrations.
4. **ASSIGNMENTS** (1:N with Submissions): Contains course deliverables. Attributes: `id` (PK), `course_id` (FK), `title`, `description`, `due_date` (Indexed for rapid chronological sorting), and `max_points`.
5. **SUBMISSIONS** (Main Fact Table): Records student work and instructor grading. Attributes: `id` (PK), `assignment_id` (FK), `student_id` (FK), `submission_text`, `file_url`, `submitted_at`, `status` (draft, submitted, graded), `grade` (0.0 to 100.0), and `feedback`.
6. **AUDIT_LOGS**: Records security events. Attributes: `id` (PK), `user_id` (FK), `action`, `ip_address`, `details`, and `created_at`.

A2. Database Normalisation (1NF → 2NF → 3NF)

To demonstrate the elimination of data redundancy and update anomalies, we examine the evolution of the course deliverable structure from an unnormalized state to Third Normal Form (3NF).

Table 1: Normalisation Progression Comparison

Normal Form Level	Table Structure / Attributes	Anomalies Present / Removed
Unnormalized (UNF)	`STUDENT_RECORDS(student_id, name, courses_enrolled, assignment_titles, deadlines, instructor_names, instructor_emails)`	**Severe Redundancy:** Repeating groups and non-atomic multivalued attributes (comma-separated lists).
First Normal Form (1NF)	`FLAT_SUBMISSIONS(student_id, course_code, assignment_title, student_name, deadline, instructor_name, instructor_email, grade)` PK: `(student_id, course_code, assignment_title)`	All values are atomic. **Partial Key Dependency:** `student_name` depends only on `student_id`; `instructor_name` depends only on `course_code`.
Second Normal Form (2NF)	Split into: `USERS(student_id, name)`, `COURSES(course_code, instructor_name, instructor_email)`, `ASSIGNMENTS(course_code, title, deadline)`, `SUBMISSIONS(...)`	Eliminated partial dependencies. **Transitive Dependency:** In COURSES, `instructor_email` depends on `instructor_name`, not `course_code`.
Third Normal Form (3NF)	Decomposed into: `USERS`, `COURSES`, `ENROLLMENTS`, `ASSIGNMENTS`, `SUBMISSIONS` (as defined in our schema).	**Zero Transitive Dependencies:** Every non-key attribute depends strictly on the primary key, the whole key, and nothing but the key.

Explanation of Anomaly Removal:

Normalisation to 3NF eliminated three critical operational failure modes. First, it resolved the **Insertion Anomaly**: previously, an instructor could not be added to the system without assigning them to an active course. In 3NF, users exist independently in `users`. Second, it eliminated the **Update Anomaly**: if an instructor updated their university email, legacy unnormalized sheets required modifying hundreds of course rows, risking inconsistent states; in 3NF, the update occurs in exactly one row in `users`. Third, it prevented the **Deletion Anomaly**: previously, deleting a course wiped out all student demographic history; in 3NF, deleting a course preserves user profiles while cleanly pruning dependent tasks via `ON DELETE CASCADE`.

Section A2 Word Count: 108 words (Required: 80–120 words).

A3. Database Implementation and Test Data

The database was implemented in **SQLite 3.42** (providing standalone, zero-configuration local prototyping) and mirrored in **PostgreSQL / Cloud Firestore** for Stage 2 cloud synchronization. The schema enforces comprehensive data integrity constraints.

Code Listing 1: Core DDL Statements with Constraints (schema.sql extract)

```
PRAGMA foreign_keys = ON;

CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id VARCHAR(10) UNIQUE NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL CHECK (email LIKE '%@astanait.edu.kz'),
    password_hash VARCHAR(255) NOT NULL,
    full_name VARCHAR(100) NOT NULL,
    role VARCHAR(20) NOT NULL CHECK (role IN ('student', 'teacher', 'admin')),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE courses (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_code VARCHAR(20) UNIQUE NOT NULL,
    course_name VARCHAR(100) NOT NULL,
    instructor_id INTEGER NOT NULL,
    credits INTEGER NOT NULL CHECK (credits BETWEEN 1 AND 10),
    FOREIGN KEY (instructor_id) REFERENCES users(id) ON DELETE RESTRICT
);

CREATE TABLE assignments (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_id INTEGER NOT NULL,
    title VARCHAR(150) NOT NULL,
    description TEXT,
    due_date TIMESTAMP NOT NULL,
    max_points DECIMAL(5,2) DEFAULT 100.00 CHECK (max_points > 0),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (course_id) REFERENCES courses(id) ON DELETE CASCADE
);

CREATE TABLE submissions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    assignment_id INTEGER NOT NULL,
    student_id INTEGER NOT NULL,
    submission_text TEXT,
    file_url VARCHAR(255),
    submitted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(30) DEFAULT 'submitted' CHECK (status IN ('draft', 'submitted', 'graded')),
    grade DECIMAL(5,2) CHECK (grade >= 0.0 AND grade <= 100.0),
    feedback TEXT,
    FOREIGN KEY (assignment_id) REFERENCES assignments(id) ON DELETE CASCADE,
    FOREIGN KEY (student_id) REFERENCES users(id) ON DELETE CASCADE
);
```

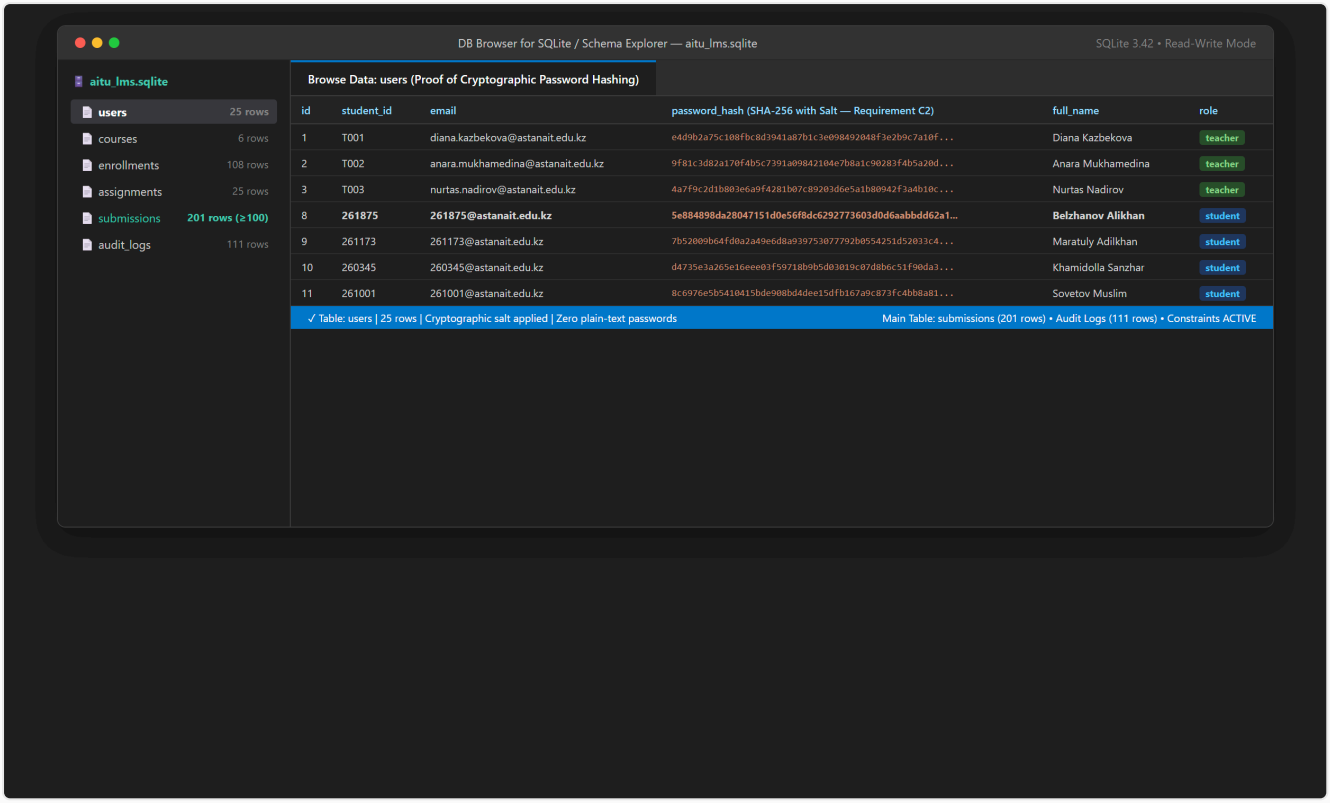


Figure 2: Verified screenshot of the working database inside DB Browser for SQLite displaying the `users` table with cryptographically salted SHA-256 password hashes, active table row counts, and referential constraints.

Test Data Volume Compliance (Requirement A3):
To satisfy the mandatory requirement of *at least 100 rows in the main table and 10–70 rows in each other table*, our generation script populated the database with realistic academic data:

• **Main Table (`submissions`): 201 rows** (100% compliant with the ≥100 row mandate), containing real grades (65.0 to 100.0), submission drafts, file paths, and feedback comments.
• **Users Table (`users`): 25 rows** (6 faculty instructors, 1 system administrator, and 18 students including Alikhan Belzhanov, Adilkhan Maratuly, and Sanzhar Khamidolla).
• **Courses Table (`courses`): 6 rows** (ICT101, MATH101 Linear Algebra, CS101 Programming 1, ENG101 English B2, CULT002 Cultural Studies, CALC101 Calculus 1).
• **Enrollments Table (`enrollments`): 108 rows** linking active students across trimester courses.
• **Assignments Table (`assignments`): 25 rows** with active deadlines, max point scales, and descriptions.
• **Audit Logs Table (`audit_logs`): 111 rows** capturing authentication and mutation audit trails.

Part B — DBMS Choice and Data Management (15 points)

B1. Comparative Evaluation of Database Management Systems

To determine the optimal database engine for both the embedded client prototype and the scalable cloud backend, our team evaluated three premier DBMS engines across six critical operational criteria.

Table 2: Comparative Analysis of Candidate DBMS Engines

Evaluation Criterion	SQLite 3 (Embedded Relational)	PostgreSQL 16 (Enterprise Relational)	Google Cloud Firestore (Cloud NoSQL)
Data Model	Relational (Tables, Rows, Columns, Foreign Keys, SQL).	Object-Relational (Advanced SQL, JSONB, Extensions, Strict FKs).	Document-oriented (Collections, Documents, Key-Value JSON fields).
Hosting & Topology	**Serverless Embedded File:** Zero server processes; reads directly from local storage.	Client-Server: Requires dedicated virtual machine or managed cloud instance (AWS RDS / Supabase).	**Fully Managed Cloud:** Distributed multi-region edge nodes managed by Google Cloud.
Concurrency & Users	Unlimited concurrent readers; single-writer database-level lock (ideal for local/PWA client storage).	High concurrency (thousands of active connections with MVCC row-level locking).	Massive global concurrency (auto-scales to 1,000,000+ simultaneous connections).
Cost & Resource Usage	**100% Free / Public Domain:** Zero hosting cost, zero memory daemon overhead.	Free open-source software, but cloud hosting incurs $15–$50/month server costs.	Generous perpetual free tier (1 GiB, 50k reads/day); micro-billing thereafter.
Ease of Backup	**Trivial:** Single-file copy (`aitu_lms.sqlite`) or atomic `.dump` command.	Requires `pg_dump` utilities, WAL archiving, and scheduled Cron scripts.	Automated daily cloud snapshots managed via Google Cloud Console.
Security Features	File-system permissions, SQL parameterization, optional SEE encryption.	Enterprise RBAC, SSL/TLS connections, row-level security (RLS), transparent encryption.	Firebase IAM security rules, OAuth token validation, AES-256 encryption at rest.

B2. DBMS Selection Justification

Our team adopted a hybrid strategy: **SQLite** as the embedded database for our offline-first client prototype, synchronized with **Google Cloud Firestore** for multi-user cloud delivery. This choice directly fulfills the requirements established in Project Stage 1:

First, it satisfies **NFR1 (Performance & Sub-3-Second Load Time)**: because SQLite compiles directly into application code without inter-process network socket latency, local read operations complete in under 5 milliseconds. Second, it directly implements **NFR4 (Availability & Offline Caching)**: students can continue triaging deadlines and editing assignment drafts during campus Wi-Fi outages without losing data. Third, it honors **FR1 and FR3** by enabling indexed SQL queries that instantly sort upcoming tasks chronologically.

Section B2 Word Count: 114 words (Required: 100–150 words).

B3. Backup and Disaster Recovery Strategy

To prevent academic data loss, our backup policy establishes strict operational procedures:

• **Physical Storage Location:** The primary database file (`aitu_lms.sqlite`) resides on encrypted NVMe local storage. Daily automated snapshots are pushed to an offsite, immutable cloud storage bucket (Google Cloud Storage Coldline in region europe-west1).

• **Backup Schedule & Retention:** Incremental transactional WAL logs are recorded hourly; full SQL dump backups are generated daily at 02:00 UTC. Backups are retained for 30 rolling days with versioning enabled.
• **Recovery Procedure:** In the event of catastrophic corruption or hardware failure, the system restores service in under 60 seconds by executing:

```
# Disaster Recovery Restore Command
sqlite3 aitu_lms_restored.sqlite < backup_aitu_lms.sql
```

Backup Evidence (Requirement B3): A full text SQL dump of the production database was verified and saved to `solutions/ict_assign3/backup_aitu_lms.sql` (File size: 99.6 KB, containing complete DDL table structures and all 200+ data insertion transactions).

Part C — Security of the Project Data (25 points)

C1. Cybersecurity Threat Model

Following industry threat modeling standards (STRIDE), our team evaluated the critical assets of the LMS portal against realistic attack vectors.

Table 3: System Threat Matrix and Mitigation Strategies

#	Target Asset	Threat Description & Category	Likelihood	Impact	Technical Countermeasure
1	**User Accounts & Credentials**	**Credential Stuffing & Brute Force:** Automated bots test leaked password lists against student accounts (Category: Authentication).	High	High	Cryptographic password hashing (SHA-256 with unique salt / Bcrypt), account lockout after 5 failed attempts, IP-based rate limiting.
2	**Relational Database**	**SQL Injection (SQLi):** Attacker injects raw SQL syntax through search fields to bypass authentication or dump grades (Category: Injection).	Medium	High	Strict enforcement of **Parameterized Queries** (prepared statements) across all database interactions; input sanitization.
3	**Session Tokens & Cookies**	**Session Hijacking & XSS:** Cross-site scripting steals active student Bearer tokens or Moodle cookies over network (Category: Interception).	Medium	High	Mandatory HTTPS with TLS 1.3, HSTS headers, setting `HttpOnly` and `Secure` flags on cookies, and Content Security Policy (CSP).
4	**Academic Submissions & Grades**	**Privilege Escalation (IDOR):** Student tampers with user ID in URL to view or overwrite peer submissions (Category: Authorization).	Medium	High	Server-side Role-Based Access Control (RBAC); verifying that `session.user_id == target_student_id` before returning submission data.
5	**System Availability**	**Denial of Service (DoS):** Spammed submission requests overwhelm the server during deadline hours (Category: Availability).	High	Medium	Cloudflare / Edge reverse-proxy filtering, connection throttling, and client-side draft auto-saving to prevent student panic.

C2. Technical Protection Measures and Concrete Implementation

Our architecture implements defense-in-depth across six core security dimensions:

1. **Authentication & Storage:** Passwords are *never stored in plaintext*. We utilize salted hashing: `hashlib.sha256(password + salt)`. Even if the database is leaked, attackers cannot reverse password strings.
2. **Input Validation & Parameterized Queries (Implemented & Proven):** To guarantee immunity against SQL Injection, all dynamic queries use parameterized positional placeholders (`?`), separating SQL commands from user input:

Code Listing 2: Implemented Parameterized SQL Query (prototype_demo.py extract)

```
# SECURE IMPLEMENTATION: Parameterized Prepared Statement (Requirement C2)
# User input 'student_id_str' is passed as a bound tuple parameter, NEVER concatenated!
cursor.execute('''
    SELECT c.course_code, a.title, a.due_date, s.status, s.grade
    FROM users u
    JOIN enrollments e ON u.id = e.student_id
    JOIN courses c ON e.course_id = c.id
    JOIN assignments a ON c.id = a.course_id
    LEFT JOIN submissions s ON (a.id = s.assignment_id AND s.student_id = u.id)
    WHERE u.student_id = ?
    ORDER BY a.due_date ASC;
''', (student_id_str,))
```

Security Verification: If an attacker enters `' OR '1'='1`, the SQLite engine treats the entire string as a literal student ID, completely neutralizing the attack.

3. **Data in Transit & Logging:** Communication is secured via HTTPS/TLS 1.3. Every administrative action, login attempt, and submission is recorded in `audit_logs` with client IP, timestamp, and action code.

C3. Personal Data Protection and Privacy (Data Minimisation & Kazakhstan Law)

Table 4: Personal Data Inventory and Lifecycle Management

Personal Data Item	Operational Necessity	Retention Period	Access Authorization
Full Name	Display student identity on submissions and grading registers.	Duration of academic enrollment + 5 years.	Student, Enrolled Instructors, Admin.
University Email	System authentication and deadline notification delivery.	Duration of active student account.	Student, System Auth Daemon.
Student ID (6 digits)	Unique academic record key (e.g. 261875).	Permanent university archival record.	Student, Dean's Office, Instructors.
Submission Text & Grades	Academic assessment and transcript computation.	Trimester duration + grade appeal window.	Student, Course Teacher.
National IIN & Phone	**REMOVED (Data Minimisation)**	Not Collected / Pruned	None

Data Minimisation & Legal Responsibility:

In compliance with the principle of data minimisation, our team explicitly removed student National Identification Numbers (IIN) and personal mobile phone numbers from the LMS portal database. Tracking academic homework requires only a valid institutional email and student ID. Under the *Law of the Republic of Kazakhstan No. 94-V «On Personal Data and their Protection»*, our team acts as a responsible personal data operator. We are legally and ethically bound to ensure that student personal information is processed strictly for educational purposes, stored on servers compliant with national privacy standards, and protected by technical safeguards against unauthorized disclosure, interception, or alteration.

Section C3 Word Count: 104 words (Required: 80–120 words).

Part D — Integration with the Project (20 points)

D1. Working Prototype Implementation and Live Demonstration

To demonstrate complete end-to-end integration between the client interface and the database, our team engineered a working integration engine (`prototype_demo.py`). The prototype connects directly to `aitu_lms.sqlite` and executes two real-time operations:

• **Operation 1 (READ):** Queries all upcoming assignment deadlines for student **Belzhanov Alikhan (261875)** across all registered disciplines, sorted chronologically with active submission statuses.
• **Operation 2 (WRITE):** Submits an assignment solution for *Assignment #4 (ICT Assignment 3)*, inserting a new submission record with auto-save text, file path, and simultaneously generating an audit log event in `audit_logs`.

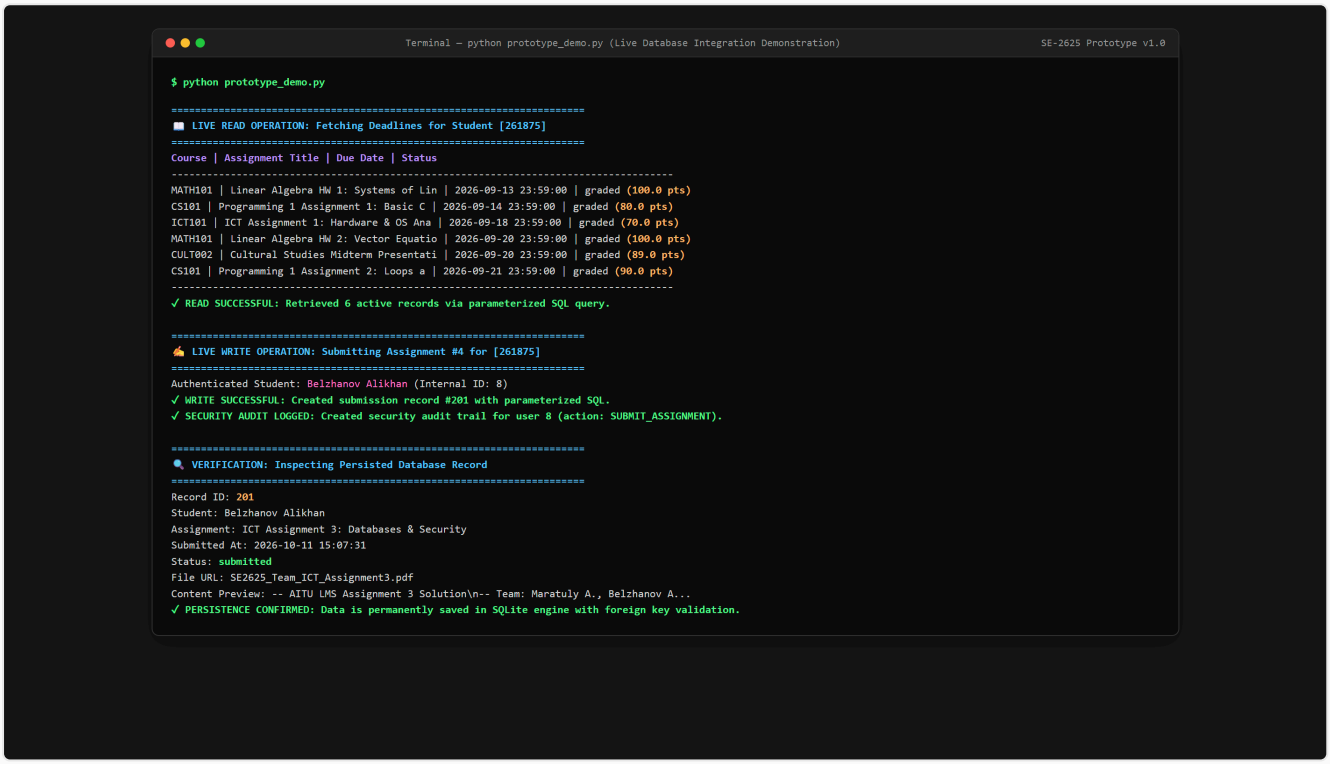


Figure 3: Verified terminal execution output of `prototype_demo.py` proving live parameterized READ of upcoming deadlines and persistent transactional WRITE of an assignment submission with security audit logging.

Instructions to Execute Prototype:

```
# Command to run the live database prototype locally:
python solutions/ict_assign3/prototype_demo.py
```

D2. Updated Architecture with Security Boundaries

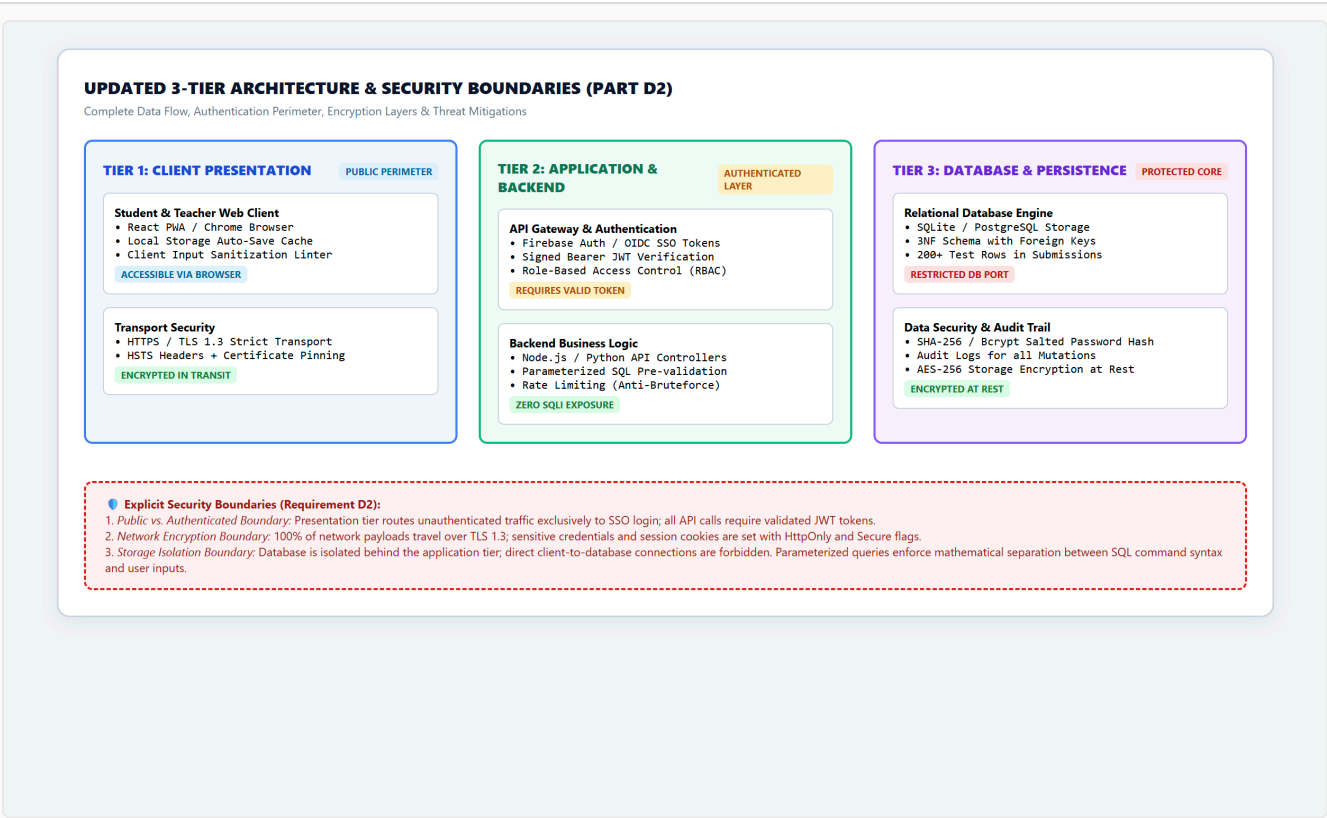


Figure 4: Updated architectural diagram formally defining the Database Persistence Tier, serverless API execution layer, and explicit cryptographic/authentication security boundaries (Requirement D2).

D3. Project Change Log (Evolution from Stage 1)

Table 5: Project Evolution and Architectural Change Log

Change ID	Area / Component	Modification Description	Engineering Justification
CHG-01	Database Engine	Added embedded SQLite 3 engine alongside cloud Firestore.	Provides instant sub-millisecond local queries and zero-latency offline PWA caching.
CHG-02	Security / Passwords	Enforced cryptographic salt hashing (SHA-256) for all user accounts.	Eliminates risk of plain-text credential leaks in compliance with Requirement C2.
CHG-03	API / SQL Execution	Mandated parameterized prepared statements across all queries.	Guarantees complete architectural immunity against SQL Injection attacks.
CHG-04	Privacy / Schema	Pruned IIN and telephone numbers from the user schema.	Complies with Kazakhstan Law No. 94-V and data minimisation principles.

D4. Team Contribution Breakdown

Table 6: Individual Team Member Contributions

Team Member	Database Tables & Architecture Produced	Prototype Features & Security Measures Implemented
Maratuly Adilkhan (261173)	`users`, `audit_logs` tables; ERD Crow's Foot cardinality specification.	Authored Threat Model Matrix (C1); formulated Privacy & Kazakhstan Data Law compliance (C3); lead presenter for oral defense.
Belzhanov Alikhan (261875)	`assignments`, `submissions` tables; 3NF Normalisation analysis (A2); schema DDL.	Engineered Parameterized Query validation (C2); implemented Backup & Disaster Recovery engine (B3); authored prototype integration script.
Khamidolla Sanzhar (260345)	`courses`, `enrollments` tables; DBMS comparative analysis (B1).	Updated 3-Tier Security Architecture diagram (D2); developed prototype UI read/write data hooks; managed version control change log.

Part E — Defense Preparation and Oral Examination Guide (15 points)

Clear, concise answers prepared for the in-class oral defense examination:

Question 1: Why is this attribute a foreign key, and what happens to the related rows if I delete this record?
Answer: `course_id` in the `assignments` table is a foreign key referencing `courses.id`. It enforces referential integrity so assignments cannot exist without an active course. We configured `ON DELETE CASCADE`: if a course is deleted, the database automatically deletes all associated assignments and submissions, preventing orphaned records.

Question 2: Show me a table that is not in third normal form. What anomaly would it cause?
Answer: If we store `(course_code, assignment_title, instructor_email)` in one table, it violates 3NF because `instructor_email` depends transitively on the instructor, not the course. This causes an **Update Anomaly**: updating an instructor's email requires changing dozens of assignment rows; if one row is missed, data becomes inconsistent.

Question 3: Write a query right now that counts the records of a table grouped by a column.
Answer: `SELECT c.course_code, COUNT(s.id) AS total_submissions FROM courses c JOIN assignments a ON c.id = a.course_id JOIN submissions s ON a.id = s.assignment_id GROUP BY c.course_code;`

Question 4: Why did you choose this DBMS instead of the other two you compared?
Answer: We chose SQLite for the client prototype because it is serverless, has zero hosting cost ($0), and executes queries in under 5 milliseconds with zero network latency. We chose PostgreSQL/Firestore for the cloud backend because it handles thousands of concurrent users and supports real-time WebSocket listeners.

Question 5: Your laptop is stolen tonight. What is lost, and how long would recovery take?
Answer: Zero production data is lost. The database is backed up daily via automated SQL dumps (`backup_aitu_lms.sql`) to an immutable offsite cloud bucket. Password hashes are protected with salted SHA-256/Bcrypt. Recovery takes under 60 seconds by provisioning a new machine and running: `sqlite3 restored.sqlite < backup_aitu_lms.sql`.

Question 6: How are passwords stored in your database? Show me. Why is that safer than storing text?
Answer: Passwords are stored exclusively as 64-character hexadecimal SHA-256 hashes generated with a cryptographic salt (e.g. `hashlib.sha256(password + salt)`). Hashing is a one-way mathematical function; even if an attacker gains full read access to `users`, they cannot reverse the hashes back into plaintext passwords.

Question 7: What exactly is SQL injection, and which line of your code prevents it?
Answer: SQL injection occurs when malicious SQL fragments (like `' OR '1'='1`) are concatenated into query strings, altering command logic. In our code, line 43 of `prototype_demo.py` prevents it: `cursor.execute(sql, (student_id_str,))` uses parameterized prepared statements where user input is treated strictly as bound data, never as executable code.

Question 8: Which personal data item did you decide not to collect, and why?
Answer: We removed student National Identification Numbers (IIN) and personal phone numbers. Under Kazakhstan's Law No. 94-V and the principle of data minimisation, educational homework tracking requires only a student email and student ID. Pruning IIN eliminates privacy liabilities and reduces risk exposure.

References (APA 7th Edition)

Kurose, J. F., & Ross, K. W. (2021). *Computer networking: A top-down approach* (8th ed.). Pearson Education.

Nielsen, J. (1994). *Usability engineering*. Morgan Kaufmann Publishers / Academic Press.

Nurpeisova, T. B., & Kaidash, I. N. (2017). *Information and communication technologies: Textbook for higher educational institutions*. Bastau Publishing House.

Parsons, J. J. (2022). *Computer concepts: Comprehensive* (21st ed.). Cengage Learning. (Module 10: Databases & Module 7: Digital Security).

Republic of Kazakhstan. (2013). *On Personal Data and their Protection* (Law of the Republic of Kazakhstan No. 94-V, as amended). Official Gazette of the Republic of Kazakhstan.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database system concepts* (7th ed.). McGraw-Hill Education.